

The Moore–Penrose Pseudoinverse

Foundations, Derivations, and Connection to SVD

July 10, 2026

Contents

1	Motivation	2
2	Definition: The Moore–Penrose Conditions	2
3	Connection to Singular Value Decomposition (SVD)	2
3.1	The SVD Factorisation	2
3.2	Full vs. Thin SVD	3
3.3	The Pseudoinverse via SVD	3
4	Three Regimes: Square, Tall, and Wide Matrices	3
4.1	Square, Full-Rank Matrix	3
4.2	Tall Matrix: Overdetermined System	3
4.3	Wide Matrix: Underdetermined System	4
5	Derivations of the Pseudoinverse (Tall Case)	4
5.1	Derivation 1: Least Squares (Calculus)	4
5.2	Derivation 2: Geometric Projection	4
5.3	Derivation 3: Tikhonov Regularisation (Limit)	5
6	Handling Rank Deficiency: Manual Removal vs. SVD	5
6.1	A Natural but Flawed Intuition	5
6.2	Why Manual Removal Fails in Practice	6
6.3	What SVD Does Instead	6
6.4	Side-by-Side Comparison	7
6.5	The Near-Rank-Deficient Case: Truncated SVD	7
7	SVD and Rank-Deficient Matrices	7
8	Summary	8

1 Motivation

A fundamental problem in linear algebra is solving the system

$$A\mathbf{x} = \mathbf{b}$$

When $A \in \mathbb{R}^{n \times n}$ is square and invertible, the unique solution is $\mathbf{x} = A^{-1}\mathbf{b}$. However, in many practical settings:

- A is **rectangular** (more equations than unknowns, or vice versa), so no square inverse exists.
- A is **rank-deficient**, making it singular even if square.
- The system is **overdetermined** (no exact solution) or **underdetermined** (infinitely many solutions).

The **Moore–Penrose pseudoinverse** A^+ generalises the matrix inverse to handle all these cases gracefully.

2 Definition: The Moore–Penrose Conditions

Definition 1. For $A \in \mathbb{R}^{m \times n}$, the **Moore–Penrose pseudoinverse** $A^+ \in \mathbb{R}^{n \times m}$ is the unique matrix satisfying all four Penrose conditions:

$$AA^+A = A \tag{1}$$

$$A^+AA^+ = A^+ \tag{2}$$

$$(AA^+)^{\top} = AA^+ \tag{3}$$

$$(A^+A)^{\top} = A^+A \tag{4}$$

Conditions (1) and (2) say A^+ acts as a “weak” two-sided inverse. Conditions (3) and (4) enforce symmetry (Hermitian) of the products AA^+ and A^+A , which ensures A^+ is an *orthogonal* projector.

Key fact: When A is square and invertible, $A^+ = A^{-1}$ recovers the ordinary inverse.

3 Connection to Singular Value Decomposition (SVD)

3.1 The SVD Factorisation

Any matrix $A \in \mathbb{R}^{m \times n}$ of rank r can be decomposed as:

$$A = U\Sigma V^{\top}$$

where:

- $U \in \mathbb{R}^{m \times m}$ — orthogonal matrix of **left singular vectors** (columns form an orthonormal basis for $\mathbb{R}^m$)
- $\Sigma \in \mathbb{R}^{m \times n}$ — diagonal matrix of **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, with zeros elsewhere

- $V \in \mathbb{R}^{n \times n}$ — orthogonal matrix of **right singular vectors** (columns form an orthonormal basis for $\mathbb{R}^n$)

Since U and V are orthogonal, $U^{-1} = U^\top$ and $V^{-1} = V^\top$, making them cheap to invert.

3.2 Full vs. Thin SVD

	U	Σ	V
Full SVD	$m \times m$	$m \times n$	$n \times n$
Thin SVD	$m \times r$	$r \times r$	$n \times r$

For computing A^+ , the **thin SVD** is sufficient — the extra columns in the full SVD correspond to zero singular values and contribute nothing.

3.3 The Pseudoinverse via SVD

Theorem 1. Given $A = U\Sigma V^\top$, the Moore–Penrose pseudoinverse is:

$$A^+ = V\Sigma^+U^\top$$

where Σ^+ is formed by taking the reciprocal of each **non-zero** singular value and leaving zeros as zero:

$$\Sigma^+ = \text{diag}\left(\frac{1}{\sigma_1}, \frac{1}{\sigma_2}, \dots, \frac{1}{\sigma_r}, 0, \dots, 0\right)^\top$$

Intuition. SVD says: rotate ($V^\top$) $\rightarrow$ scale (Σ) $\rightarrow$ rotate (U). The pseudoinverse simply reverses this: rotate back ($U^\top$) $\rightarrow$ invert the scaling (Σ^+) $\rightarrow$ rotate back (V). The orthonormality of U and V makes this clean reversal possible.

Why zeros stay zero. Rank-deficient directions (zero singular values) carry no information about A — they correspond to the null space. Trying to invert them would be division by zero, and setting them to zero is exactly what discards these redundant directions, mirroring the intuition of “removing linearly dependent columns.”

4 Three Regimes: Square, Tall, and Wide Matrices

4.1 Square, Full-Rank Matrix

$A \in \mathbb{R}^{n \times n}$, rank n . A unique exact solution exists:

$$\mathbf{x} = A^{-1}\mathbf{b} = A^+\mathbf{b}$$

4.2 Tall Matrix: Overdetermined System

$A \in \mathbb{R}^{m \times n}$ with $m > n$, full column rank. More equations than unknowns — no exact solution in general. The pseudoinverse gives the **least-squares solution**:

$$\mathbf{x}^+ = A^+\mathbf{b} = (A^\top A)^{-1}A^\top \mathbf{b} \quad \text{minimises } \|A\mathbf{x} - \mathbf{b}\|^2$$

4.3 Wide Matrix: Underdetermined System

$A \in \mathbb{R}^{m \times n}$ with $m < n$, full row rank. More unknowns than equations — infinitely many exact solutions exist. The pseudoinverse gives the **minimum-norm solution**:

$$\mathbf{x}^+ = A^+ \mathbf{b} = A^\top (AA^\top)^{-1} \mathbf{b} \quad \text{minimises } \|\mathbf{x}\|$$

Case	Shape	Solutions	A^+ formula	Minimises
Square (full rank)	$n \times n$	Exactly one	A^{-1}	—
Tall (full col rank)	$m \times n, m > n$	None (exact)	$(A^\top A)^{-1} A^\top$	$\ A\mathbf{x} - \mathbf{b}\ $
Wide (full row rank)	$m \times n, m < n$	Infinitely many	$A^\top (AA^\top)^{-1}$	$\ \mathbf{x}\ $
Rank-deficient	Any	None or ∞	$V\Sigma^+U^\top$	Both

5 Derivations of the Pseudoinverse (Tall Case)

We focus on the tall, full-column-rank case where $A^+ = (A^\top A)^{-1} A^\top$.

5.1 Derivation 1: Least Squares (Calculus)

Goal: Minimise $f(\mathbf{x}) = \|A\mathbf{x} - \mathbf{b}\|^2$.

Expanding:

$$f(\mathbf{x}) = \mathbf{x}^\top A^\top A \mathbf{x} - 2\mathbf{x}^\top A^\top \mathbf{b} + \mathbf{b}^\top \mathbf{b}$$

Differentiating and setting to zero:

$$\nabla_{\mathbf{x}} f = 2A^\top (A\mathbf{x} - \mathbf{b}) = \mathbf{0}$$

This yields the **normal equations**:

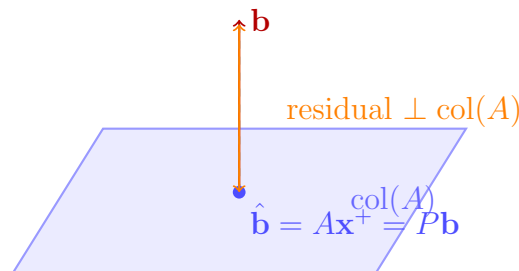
$$A^\top A \mathbf{x} = A^\top \mathbf{b}$$

Since A has full column rank, $A^\top A$ is $n \times n$ and invertible. Solving:

$$\mathbf{x}^+ = (A^\top A)^{-1} A^\top \mathbf{b} \implies A^+ = (A^\top A)^{-1} A^\top$$

5.2 Derivation 2: Geometric Projection

Column space perspective. When $A\mathbf{x} = \mathbf{b}$ has no exact solution, $\mathbf{b}$ lies outside the column space $\text{col}(A)$. The best we can do is find $\mathbf{x}$ such that $A\mathbf{x}$ is the **closest point** in $\text{col}(A)$ to $\mathbf{b}$.



The closest point $\hat{\mathbf{b}} \in \text{col}(A)$ to $\mathbf{b}$ is its **orthogonal projection** $\hat{\mathbf{b}} = P\mathbf{b}$, where P is the orthogonal projector onto $\text{col}(A)$. The residual $\mathbf{b} - \hat{\mathbf{b}}$ is perpendicular to every column of A :

$$A^\top(\mathbf{b} - A\mathbf{x}^+) = \mathbf{0} \iff A^\top A\mathbf{x}^+ = A^\top \mathbf{b}$$

which is exactly the normal equations again — the geometric and calculus views are equivalent.

The projection matrix. The projector onto $\text{col}(A)$ is:

$$P = A(A^\top A)^{-1}A^\top$$

and satisfies $P^2 = P$ (idempotent) and $P^\top = P$ (symmetric). Since $P = AA^+$, we read off:

$$A^+ = (A^\top A)^{-1}A^\top$$

Key identity: AA^+ is the orthogonal projector onto the **column space** of A .
Key identity: A^+A is the orthogonal projector onto the **row space** of A .

5.3 Derivation 3: Tikhonov Regularisation (Limit)

Add a regularisation term and solve:

$$\min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|^2 + \lambda\|\mathbf{x}\|^2, \quad \lambda > 0$$

The unique closed-form solution is:

$$\mathbf{x}_\lambda = (A^\top A + \lambda I)^{-1}A^\top \mathbf{b}$$

$(A^\top A + \lambda I)$ is always invertible for $\lambda > 0$, even when $A^\top A$ is singular. The pseudoinverse emerges as the limit:

$$A^+ = \lim_{\lambda \rightarrow 0^+} (A^\top A + \lambda I)^{-1}A^\top$$

This connects A^+ directly to **ridge regression** in machine learning ($\lambda > 0$ controls regularisation), and shows the pseudoinverse is numerically approximable in a stable way.

6 Handling Rank Deficiency: Manual Removal vs. SVD

6.1 A Natural but Flawed Intuition

Suppose $A \in \mathbb{R}^{m \times n}$ is tall but rank-deficient, with rank $r < n$. Some columns are linearly dependent — they carry no new information. A natural idea is:

1. Identify and **remove** the redundant (linearly dependent) columns.
2. This yields a smaller matrix $\tilde{A} \in \mathbb{R}^{m \times r}$ of full column rank.
3. Apply the left pseudoinverse: $\tilde{A}^+ = (\tilde{A}^\top \tilde{A})^{-1}\tilde{A}^\top$.
4. Solve and map the solution back to the original coordinates.

This approach is *valid in spirit* — the rank- r subspace is genuinely what matters. However, it runs into serious practical difficulties.

6.2 Why Manual Removal Fails in Practice

Problem 1: Which columns do you drop?

In clean textbook examples, a column may be exactly twice another. In real data, no two columns are ever *exactly* dependent — they are only *nearly* dependent due to measurement noise, floating-point errors, or correlation. This makes the keep/drop decision inherently ambiguous:

$$\mathbf{a}_3 = 2\mathbf{a}_1 + \varepsilon, \quad \|\varepsilon\| \ll 1$$

Is $\mathbf{a}_3$ redundant? The answer depends on ε in a way that has no clean threshold. A hard binary decision (keep or drop) is numerically fragile.

Problem 2: You lose track of the original variables.

After dropping columns, $\tilde{A}$ operates on a reduced set of unknowns $\tilde{\mathbf{x}} \in \mathbb{R}^r$, not the original $\mathbf{x} \in \mathbb{R}^n$. The solution $\tilde{\mathbf{x}}^+$ does not directly answer what the dropped unknowns should be. Typically one assigns them the value zero by convention, but this is an arbitrary choice with no optimality guarantee.

Problem 3: Mapping back is ambiguous.

Even after solving in the reduced space, reconstructing a full $\mathbf{x} \in \mathbb{R}^n$ requires an explicit bookkeeping of which columns were dropped and a decision about how to fill in the missing entries. This is error-prone and not unique.

6.3 What SVD Does Instead

SVD performs the *same conceptual step* — discarding redundant directions — but does so **automatically, optimally, and in a coordinate system perfectly adapted to A** .

Step 1: Rotate into a new basis (via $V^\top$).

Instead of working in the original column coordinates, SVD rotates into the basis of *right singular vectors*. In this rotated space, the directions are ordered by how much variance / information they carry (the singular values $\sigma_1 \geq \dots \geq \sigma_r > 0$).

Step 2: Identify redundant directions (via Σ).

The zero singular values *exactly* identify the null-space directions — no ambiguous thresholding required in the exact case:

$$\Sigma = \text{diag}(\underbrace{\sigma_1, \dots, \sigma_r}_{\text{keep}}, \underbrace{0, \dots, 0}_{\text{discard}})$$

Step 3: Invert only the kept directions (via Σ^+).

$$\Sigma^+ = \text{diag}\left(\frac{1}{\sigma_1}, \dots, \frac{1}{\sigma_r}, 0, \dots, 0\right)^\top$$

Setting the reciprocals of zero singular values to zero (rather than ∞) is the mathematically principled choice: it produces the minimum-norm solution among all least-squares minimisers.

Step 4: Rotate back automatically (via V).

Because V is orthogonal, the rotation is trivially reversed. The solution is returned in the *original* coordinate space with no bookkeeping:

$$\mathbf{x}^+ = V\Sigma^+U^\top \mathbf{b}$$

6.4 Side-by-Side Comparison

Step	Manual removal	SVD pseudoinverse
Find redundancy	Inspect columns (fragile)	Non-zero σ_i (automatic)
Handle near-dependency	Hard threshold (arbitrary)	Soft via $\sigma_i \approx 0$
Discard redundancy	Delete columns (loses variables)	Set $1/\sigma_i = 0$ in Σ^+
Solve reduced system	In smaller $\tilde{\mathbf{x}}$ space	In rotated space via $U^\top$
Map back	Assign zeros (arbitrary)	V rotates back automatically
Optimality of result	Not guaranteed	Minimum norm, guaranteed

Takeaway. The intuition of “remove linearly dependent columns” is correct, but executing it manually requires arbitrary choices at every step. SVD realises the same idea automatically, in an optimally adapted coordinate system, with a guaranteed minimum-norm result and no bookkeeping. Manual removal is SVD done poorly; SVD is manual removal done perfectly.

6.5 The Near-Rank-Deficient Case: Truncated SVD

SVD also handles the practically important case where A is *not* exactly rank-deficient but has very small singular values. A **truncated SVD** sets all $\sigma_i < \epsilon$ to zero before inverting:

$$\sigma_i^+ = \begin{cases} 1/\sigma_i & \text{if } \sigma_i \geq \epsilon \\ 0 & \text{if } \sigma_i < \epsilon \end{cases}$$

This is precisely the principled version of the “remove near-dependent columns” idea — and it is numerically stable where manual removal is not.

7 SVD and Rank-Deficient Matrices

When A is **not** full rank, neither $(A^\top A)^{-1}A^\top$ nor $A^\top(AA^\top)^{-1}$ is well-defined. SVD handles this seamlessly.

What SVD does geometrically.

1. **Rotate** the input space via $V^\top$, aligning coordinates with singular directions.
2. **Scale** along each direction by σ_i . Directions with $\sigma_i = 0$ are the null space.
3. **Rotate** the output space via U .

Σ^+ inverts only the non-zero singular values, discarding null-space directions entirely. This is equivalent to the intuition of “removing linearly dependent columns,” but done automatically and optimally in a rotated coordinate system.

Numerical stability. In practice, singular values may be very small rather than exactly zero (due to noise). A **truncated SVD** sets all singular values below a threshold ϵ to zero before inverting, yielding a robust numerical pseudoinverse.

8 Summary

Derivation approach	Core perspective
Least squares (calculus)	A^+ is an optimiser: minimise $\ A\mathbf{x} - \mathbf{b}\ ^2$
SVD	A^+ is a structured inverse: $V\Sigma^+U^\top$
Moore–Penrose axioms	A^+ is the unique solution to four algebraic conditions
Projection geometry	A^+ maps $\mathbf{b}$ to the nearest point in $\text{col}(A)$
Tikhonov regularisation	A^+ is the limit of ridge regression as $\lambda \rightarrow 0$

Unifying intuition. The Moore–Penrose pseudoinverse always finds the “most sensible” solution to $A\mathbf{x} = \mathbf{b}$: when no solution exists it finds the *best approximate* one; when infinitely many exist it finds the *simplest* one; and via SVD it does all of this stably even when A is rank-deficient.